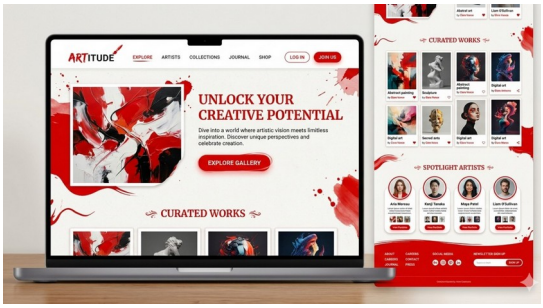
Autonomous Brand Identity & Competitive Intelligence Agent



Revised build-ready specification for AI agents, frontend, and UI/UX

Theme: artistic red and white
Inspired by the provided visual reference

Author: David
Platform: Google Antigravity IDE
Version: 2.0
Date: 2026-06-12

1. Executive Summary

This document is a complete revision of the original plan. It closes the architectural gaps, adds explicit schemas, defines the LangGraph state machine, and extends the product plan with a full frontend and UI/UX specification.

The target system is not a generic chatbot. It is an evidence engine for brand teams: it ingests internal brand material, classifies user requests, retrieves the relevant brand rules, analyzes competitor visuals, validates the result against guardrails, and outputs a structured report that agents can execute on without ambiguity.

The final product should feel editorial, premium, and visually confident: red as the primary accent, white as the dominant surface, and disciplined typography with strong hierarchy. That same visual language should carry through the frontend, report templates, and exported PDF outputs.

2. What Was Fixed

- **Embedding model specification:** Added a configurable embedding strategy, dimension policy, and versioning plan.
- **Structured output schema:** Defined formal JSON schemas for routing, image analysis, synthesis, validation, and export.
- **Image analysis format:** Restricted the vision node to observable properties with confidence and failure states.
- **LangGraph state definition:** Introduced a typed state object, node contracts, and stop conditions.
- **Query routing / classification:** Added deterministic routing rules and fallback handling for ambiguity.
- **Chunking strategy:** Replaced vague slicing with a section-aware hybrid chunking pipeline.
- **Evaluation / validation:** Added offline and online validation, scoring, and regression gates.
- **UI/UX + frontend:** Added a complete frontend architecture, pages, components, and interaction model.
- **Caching, logging, cost control, errors, testing, re-indexing:** Added operational controls so the system can run safely in production and under agentic workflows.

3. Product Definition

Purpose: help brand owners, strategists, designers, and AI agents compare internal brand guidelines against competitor assets such as logos, landing pages, banners, social posts, color systems, and tone-of-voice samples.

Primary output: a source-grounded, structured report containing evidence, visual observations, gaps, risks, and recommendations.

Operating principle: when evidence is insufficient, the system must say so clearly instead of inventing a confident answer.

4. Revised System Architecture

The system should be built as a layered pipeline with narrow, testable interfaces. Each layer has one responsibility and emits structured output for the next layer.

Layer	Role
Ingestion	Loads PDFs, Markdown, images, and optional future web captures. Normalizes text and metadata.
Chunking	Produces section-aware, semantically coherent text chunks with traceable IDs.
Embedding	Generates vector embeddings with a versioned, configurable model.
Retrieval	Finds the most relevant internal brand evidence using hybrid lexical + vector search.
Routing	Classifies a request as text-only, image-only, or multimodal.
Vision	Extracts observable features from competitor assets, never speculative brand strategy.
Synthesis	Merges evidence into a final report with strict section schema.
Validation	Checks for missing evidence, unsupported claims, schema drift, and unsafe outputs.
Presentation	Renders the output in Markdown, the web UI, and final PDF exports.

5. Reference Technology Stack

Area	Choice
Language	Python 3.11+
Orchestration	LangChain + LangGraph
Vector store	Local persistent Chroma
Embedding	Configurable provider via environment variables
Vision / multimodal	Gemini via the official Google GenAI SDK
PDF ingestion	pypdf + layout-aware cleaning
Frontend	React + TypeScript + Tailwind + shadcn/ui
State	Typed application state, persisted run logs, and deterministic IDs
Exports	Markdown, PDF, JSON, and downloadable evidence bundles

Model choice should never be hardcoded. The embedding model, vision model, and synthesis model should all be environment-driven so the system can switch between high-quality and low-latency modes without code changes.

6. Embedding Model Specification

The embedding layer must be explicitly defined, because retrieval quality and index stability depend on it.

- Embedding model name must come from configuration, not source code.
- Store the embedding model identifier and version inside index metadata.
- Use a single embedding model per index to avoid incompatible vector spaces.
- Record the embedding dimension and provider in the collection metadata.
- Index rebuilds must be triggered whenever the embedding model changes.

```
EMBEDDING_MODEL = {
  "provider": "configurable",
  "model_name": "env:EMBEDDING_MODEL_NAME",
  "dimension": "env:EMBEDDING_DIMENSION",
  "version": "env:EMBEDDING_MODEL_VERSION",
  "index_compatibility_key": "provider:model_name:dimension"
}
```

7. Text Normalization and Chunking Strategy

The original plan mentioned chunking, but it did not define how. The revised pipeline uses a hybrid strategy so the index preserves meaning and can be debugged.

- Extract text with page numbers, section headings, and source-file provenance.
- Remove boilerplate such as repeated footers, legal headers, and OCR noise.
- Preserve headings as metadata so chunks can be grouped by section.
- Prefer structure-aware splitting by headings and paragraphs.
- Use semantic fallback chunking for long paragraphs or unstructured PDFs.
- Keep chunk sizes within a target token band, with overlap only when needed.
- Attach document version, page range, section title, and content hash to every chunk.

```
Chunk metadata:
- chunk_id
- document_id
- source_file
- document_type
- section_title
- page_start / page_end
- content_hash
- chunk_version
- tags[]
```

8. Structured Output Contracts

Every model-facing stage must emit schema-constrained data. Free-form text is only allowed in the final render step.

Object	Required fields
RoutingDecision	request_type, intent, evidence_needs, confidence, missing_inputs
VisionObservation	asset_name, observable_features,

	dominant_colors, confidence, failure_reason
RetrievalEvidence	chunk_id, source_file, page_number, section_title, score
FinalReport	overview, evidence, findings, gaps, risks, recommendations, limitations

Structured output should be validated twice: once at generation time with schema enforcement, and once in a separate validation node before the report is shown to the user.

9. Vision / Image Analysis Output Format

The vision layer must extract only observable properties. It must not guess strategy, intent, or business meaning unless the model can ground it in visible evidence.

```
{
  "asset_name": "competitor_banner_01.png",
  "asset_type": "logo|banner|landing_page|social_post|unknown",
  "observations": {
    "layout_style": "centered / split / editorial / grid / card-based",
    "hierarchy": "primary headline dominates, secondary CTA is smaller",
    "typography_style": "serif / sans-serif / mixed / decorative / condensed",
    "visual_tone": "premium / energetic / minimal / luxurious / playful",
    "whitespace_balance": "low / medium / high",
    "composition": "symmetrical / asymmetrical / layered / modular",
    "dominant_colors": [
      { "hex": "#B11116", "weight": 0.46, "approximate": true },
      { "hex": "#FFFFFF", "weight": 0.31, "approximate": true }
    ],
    "confidence": "low|medium|high"
  },
  "failure_reason": null
}
```

If the image is unreadable, corrupted, or too ambiguous, the node must return a failure state rather than a guess.

10. LangGraph State Definition and Routing Logic

The graph should remain intentionally small. The goal is reliable orchestration, not recursive autonomy.

```
class GraphState(TypedDict, total=False):
    request_id: str
    user_query: str
    attached_images: list[str]
    request_type: Literal["text", "image", "multimodal"]
    intent: Literal["brand_rules", "visual_compare", "audit", "unknown"]
    retrieval_hits: list[RetrievalEvidence]
    visual_observations: list[VisionObservation]
    synthesis_draft: str
    final_report: FinalReport
    validation_status: Literal["pass", "warn", "fail"]
    missing_inputs: list[str]
    loop_count: int
    recursion_budget: int
    errors: list[str]
```

- Router node: classify the request using keywords, asset presence, and confidence thresholds.

- Retriever node: fetch the most relevant internal brand evidence.
- Vision node: analyze uploaded competitor assets.
- Synthesis node: merge evidence into the final report.
- Validation node: reject unsupported claims and missing evidence.
- Stop condition: return a report or a hard fallback message.
- Recursion limit: small and explicit, with a bounded retry path only when new evidence appears.

Request pattern	Routing action
Text-only	Tone, strategy, positioning, guidelines, brand rules, policy, identity system.
Image-only	Logo, banner, palette, composition, layout, typography, visual style, upload.
Multimodal	Any query containing both internal rules and uploaded assets.
Ambiguous	Retrieval first, then ask for the missing asset or question detail.

11. Evaluation and Validation Layer

The validation layer is mandatory. It is the line between a prototype and a system that people can trust.

- Schema validation: ensure every object matches the expected contract.
- Evidence validation: every claim in the final report must cite a retrieved chunk or vision observation.
- Coverage validation: flag when the report is missing one or more required sections.
- Fallback validation: ensure the system uses the safe fallback when data is missing.
- Regression validation: compare routing and retrieval behavior against golden test cases.
- Quality scoring: optionally score relevance, completeness, and grounding.

A report should never be marked complete if it contains unsupported assertions, invented brand rules, or uncited visual conclusions.

12. Caching, Cost Control, and Performance

Control	Policy
Caching	Cache embeddings, retrieval results, model metadata, and repeated router decisions keyed by request hash.
Cost control	Use low-latency models for routing and extraction where possible; reserve expensive models for final synthesis.
Batching	Analyze multiple images in one request when the provider supports it.
Debounce	Avoid re-indexing and re-embedding unchanged documents.
Timeouts	Set explicit per-node timeouts and fail soft

	when a lower-priority step is unavailable.

13. Error Handling Strategy

- Missing API key: fail fast with a readable setup error.
- Unreadable document: quarantine the file and continue processing the rest.
- Unreadable image: return a structured failure state.
- No retrieval hit: return an empty result and the limitation in the final report.
- Schema mismatch: retry once with a smaller prompt; otherwise fail safely.
- Graph overrun: terminate the run and emit a bounded fallback response.

14. Logging and Observability

- Log request ID, timestamps, route decision, retrieved chunk IDs, and image filenames.
- Capture per-node latency and model usage cost.
- Store the final report with evidence references for debugging and audits.
- Keep a lightweight run manifest so a single output can be reconstructed later.
- Emit structured logs in JSON rather than free-form console text whenever possible.

15. Testing Strategy

Test type	Coverage
Unit	Loader, chunker, metadata extraction, query normalization, router classification.
Integration	Index build, retrieval hit, image analysis, multimodal synthesis.
Negative	Empty corpus, missing API key, unreadable image, no retrieval hits, corrupted PDF.
Regression	Golden queries that must route the same way after refactors.
Contract	Schema conformance for all intermediate outputs.

Recommended golden cases include a brand color mention, a competitor palette mismatch, a voice-analysis request with no voice guide, and an image-only upload that should still trigger the vision path.

16. Data Versioning and Re-indexing Plan

- Every source file gets a stable document ID and content hash.
- Track source version, embedding model version, and chunking version in index metadata.
- Rebuild the collection when any of those version keys changes.
- Never overwrite raw source files during ingestion.
- Keep prior index snapshots until the new one passes validation.

- Support deterministic re-indexing from the same source bundle.

17. CLI and Developer Interface

The project needs a minimal but explicit command surface so agents and humans can run the same pipeline.

```
python main.py ingest --source ./data
python main.py query --text "brand color hierarchy"
python main.py analyze --image competitor_banner.png
python main.py run --query "..." --image ...
python main.py export --request-id <id> --format pdf
python main.py validate --golden-set ./fixtures/golden_cases.json
```

The CLI should print a concise summary, write a structured JSON artifact, and optionally produce the PDF report.

18. UI / UX and Frontend Plan

This is the missing product layer. The frontend should make the system feel like a premium editorial intelligence tool rather than a generic chatbot.

18.1 Visual Direction

- Primary palette: deep artistic red, white, warm gray, and restrained charcoal text.
- Use large typographic hierarchy, generous whitespace, and sharp card edges softened with subtle shadow.
- Use the supplied reference image language: bold hero composition, red brush-like accents, and clean white surfaces.
- Avoid clutter, neon gradients, and overly playful iconography.
- Every page should feel like a curated art publication mixed with a professional intelligence dashboard.

18.2 Information Architecture

Page	Purpose
Dashboard	Recent analyses, status cards, index health, and quick actions.
New Analysis	Upload images, enter query, choose mode, run analysis.
Evidence Explorer	Browse retrieved chunks, page previews, citations, and confidence.
Reports	Rendered outputs, downloadable PDF/JSON, comparison history.
Settings	Model selection, cache controls, data sources, API keys, theme controls.

18.3 Core User Flows

- Upload internal brand documents and build or refresh the index.

- Paste a competitor review question and attach one or more assets.
- Review the router decision before the system runs, when desired.
- Inspect evidence cards side by side with visual observations.
- Export the final report as PDF or shareable JSON.

18.4 Key Screens

Screen	Contents
Landing / Dashboard	Large hero area, short product statement, recent runs, and a prominent red primary CTA.
Analysis Workspace	Left panel for prompt and uploads, center canvas for evidence, right rail for status and validation.
Evidence Detail Drawer	Shows source chunk text, page preview, confidence, and citation metadata.
Report Preview	Formatted final report with collapsible evidence traces.
Settings / Admin	Model toggles, budget caps, re-index controls, and log export.

18.5 Component Inventory

- Top navigation with logo, workspace selector, run status, and user menu.
- Hero card with analysis CTA and concise trust statement.
- Upload dropzone with file validation and progress states.
- Route badge that explains why the system chose text, image, or multimodal processing.
- Evidence cards with source, page, similarity score, and excerpt.
- Image observation cards with dominant colors and observable properties.
- Validation banner with pass / warn / fail states.
- Timeline or run log panel for step-by-step execution trace.

18.6 UX Rules

- Never hide the evidence path behind the final answer.
- Always show whether the result is complete, partial, or limited.
- Prefer progressive disclosure: summary first, evidence next, raw details last.
- Make failure states actionable, not cryptic.
- Allow users to copy the final Markdown, export PDF, and download JSON in one click.

18.7 Frontend Tech Stack

Area	Choice
Framework	React + TypeScript
Styling	Tailwind CSS with shadcn/ui primitives
State	React Query or equivalent for server state; local UI state for drafts
Charts / status	Lightweight component library only when necessary

PDF preview	Embedded rendered report view or downloadable artifact
Icons	Minimal and restrained; avoid decorative excess

18.8 Frontend Data Flow

```
User action -> Frontend request -> API route -> Orchestrator
-> Retrieval / vision / synthesis
-> Validation -> Stored run artifact
-> Frontend renders:
   - status
   - evidence
   - findings
   - export links
   - logs
```

19. File Structure

```
project-root/
  .env
  .env.example
  .gitignore
  requirements.txt
  main.py
  data/
    brand_guidelines/
    competitor_assets/
    samples/
  src/
    config.py
    ingestion/
      loader.py
      chunker.py
      embeddings.py
      chroma_store.py
      build_index.py
    retrieval/
      retriever.py
      query_normalizer.py
    vision/
      analyzer.py
      schemas.py
    agents/
      router.py
      synthesizer.py
      state.py
      prompts.py
      validator.py
    evaluation/
      tests.py
      fixtures.py
    ui/
      api/
      components/
      pages/
      styles/
    utils/
      logging.py
      file_io.py
```

```
    cache.py
    errors.py
```

20. Implementation Milestones

1. Phase 1: scaffold project, configuration, secrets handling, and base CLI.
2. Phase 2: ingest sources, normalize text, and build persistent Chroma index.
3. Phase 3: implement retrieval and evidence formatting.
4. Phase 4: implement structured image analysis.
5. Phase 5: wire LangGraph routing, synthesis, and validation.
6. Phase 6: add caching, logging, cost controls, and error handling.
7. Phase 7: build the frontend workspace and report viewer.
8. Phase 8: add tests, golden fixtures, and re-indexing logic.
9. Phase 9: run end-to-end validation and freeze the release candidate.

21. Definition of Done / Acceptance Criteria

- The system can ingest a brand corpus, answer a text query, analyze an image, and produce a multimodal report.
- Every final claim is grounded in either retrieved text or visual observation.
- Missing evidence results in a limitation, not a hallucination.
- The frontend supports upload, query, evidence inspection, report preview, and export.
- The system is reproducible from source plus documented environment variables.
- Tests cover routing, retrieval, image analysis, validation, and failure states.
- Re-indexing with the same inputs produces stable IDs and compatible results.

22. Final Operating Principle

Build this as a disciplined evidence system. Local Chroma stores brand memory, Gemini extracts structured multimodal observations, LangGraph routes the work, and the validator blocks unsupported claims. The frontend should reveal that process clearly, with the same red-and-white editorial identity as the reference image.

This version is ready to hand to AI agents and implementation teammates without the missing pieces that would otherwise cause drift, ambiguity, or hallucination.